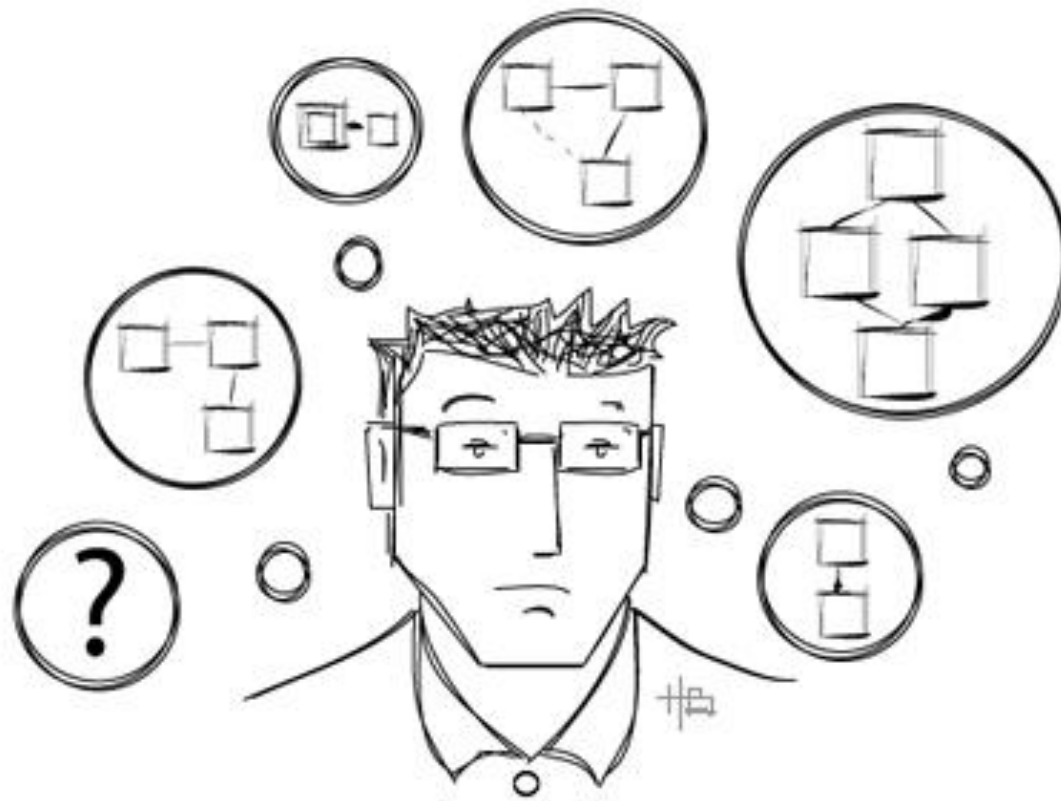


PATRONES DE DISEÑO DE SOFTWARE: INTRODUCCIÓN

ING. ZOILA RUIZ, PHD





INTRODUCCION

Los patrones de diseño definen la solución a un problema que puede ocurrir una y otra vez en el desarrollo de software, generando de esta manera, una misma solución que puede ser aplicada las veces que sea necesaria.

Son soluciones probadas y reutilizadas, ayuda a mejorar la calidad del código, fomentar la reutilización y facilitan el mantenimiento.

CLASIFICACIÓN DE PATRONES

Existen patrones independientes del dominio y otros específicos a un dominio concreto.

Patrones independientes de dominio: son generales y se aplican a cualquier tipo de aplicación

Ej: Los patrones del GoF son independientes del dominio

Patrones de Diseño de Software

Patrones específicos del dominio: son diseñados para un tipo particular de aplicación o entorno empresarial. Están fuertemente ligados al dominio funcional del software.

Ej.: Los patrones aplicables al desarrollo de aplicaciones empresariales, bancarias, médicas, educativas. Etc.

- **Domain Model:** Organización lógica del negocio dentro del código.
- **Service Layer:** Capa intermedia para coordinar la lógica del negocio.
- **Repository:** Acceso abstracto a la base de datos.
- **Unit of Work:** Manejo de transacciones de manera eficiente.
- **DTO (Data Transfer Object):** Para transferir datos entre capas.

CLASIFICACIÓN DE PATRONES

Tipo de patrón	Ámbito de aplicación	Ejemplo de uso
Independiente del dominio	Cualquier tipo de aplicación	Usar Observer para un sistema de notificaciones
Específico del dominio	Aplicaciones empresariales	Usar Repository para manejar datos de clientes en una tienda online

CLASIFICACIÓN DE PATRONES SEGÚN *Pattern-Oriented Software Architecture (POSA)*

Es una clasificación ampliada de los patrones según "Pattern-Oriented Software Architecture" (POSA). Esta clasificación es más completa y estructurada que la clásica de los GoF, y distingue entre distintos niveles de abstracción de los patrones, divididos en tres grandes categorías:

Patrones Arquitectónicos (Architectural Patterns): Los patrones arquitectónicos definen la estructura global del sistema. Se centran en la organización de subsistemas, componentes principales y sus interacciones. Son útiles en las primeras fases del diseño.

Patrones de Diseño (Design Patterns): Los patrones de diseño abordan la organización e interacción entre clases y objetos en un contexto más limitado que los arquitectónicos. Son útiles durante el diseño detallado y se centran en la reutilización y flexibilidad.

Patrones de Idioma (Idiomatic Patterns / Idioms): Los patrones de idioma se centran en cómo implementar ciertos conceptos de diseño en un lenguaje de programación específico. Traducen patrones más generales al código efectivo.

POSA

Patrones arquitectónicos	Patrones de diseño	Patrones de idioma
Aconsejan la arquitectura global que debe seguir una aplicación Ej.: El patrón Model-View-Controller (MVC) aconseja la arquitectura global que debe tener una aplicación interactiva	Explican cómo resolver un problema concreto de diseño El patrón “Data Access Object” (DAO) permite abstraer y encapsular los accesos a un repositorio de datos	Explican cómo resolver un problema particular de implementación con una tecnología concreta ¿Cómo comparar objetos correctamente en Java? Correcta redefinición de equals() y hashCode()

POSA

Característica	Arquitectónicos	De Diseño	De Idioma
Nivel	Alto (estructura global)	Medio (organización interna)	Bajo (implementación específica)
Aplicación	Arquitectura del sistema	Diseño de componentes/ clases	Código fuente concreto
Dependencia del lenguaje	No	No	Sí
Fase del desarrollo	Diseño arquitectónico	Diseño detallado	Codificación

PATRONES ORIENTADOS A OBJETOS

- ❖ “Un patrón es una regla con tres partes que expresa relación entre un determinado contexto, un problema y una solución” **Christopher Alexander** (arquitecto)
- ❖ “Un patrón describe un problema que ocurre de forma iterativa en nuestro entorno, también describe la solución a dicho problema de tal forma que podemos usar la misma solución todas las veces que sea necesario.”
- ❖ “Diseñar software orientado a objetos es difícil, y diseñar software orientado a objetos reutilizables es todavía más difícil..... Y un software capaz de evolucionar tiene que ser reutilizable.” **The Gang of Four (GoF)**

Nota: Las descripciones de patrones suelen ser independientes de los detalles de implementación o del lenguaje de programación.

PATRONES ORIENTADOS A OBJETOS

Los patrones de diseño (design patterns) son soluciones estándar y reutilizables a problemas comunes que surgen en el diseño de software. Sirven como soluciones universales a problemas comunes en una amplia gama de instancias y dominios.

Es una descripción sistemática o plantilla para resolver problemas específicos, adaptable a las necesidades de diferentes situaciones. Proporciona un conjunto compartido de conocimientos y una plataforma común para la comunicación entre desarrolladores.

Cada patrón es como un plano que se puede personalizar para resolver un problema de diseño particular de tu código. No te dan el código exacto, pero muestran como estructurar el código para que sea más fácil de construir y mantener.

Definen un lenguaje común que ayuda a tu equipo a comunicarse con más eficiencia.

PATRONES ORIENTADOS A OBJETOS

Objetivos

- ❖ Proporcionar catálogos de elementos reusables en el diseño de software
- ❖ Evitar la reiteración en la búsqueda de soluciones a problemas ya conocidos y solucionados anteriormente.
- ❖ Formalizar un vocabulario común entre diseñadores ? Estandarizar el modo en que se realiza el diseño
- ❖ Facilitar el aprendizaje a las nuevas generaciones de diseñadores condensando conocimiento ya existente.

Nota: No es obligatorio utilizar patrones.

PATRONES ORIENTADOS A OBJETOS

Elementos básicos de un patrón

1. **Contexto:** situación en la que se presenta el problema de diseño.
2. **Problema:** descripción del problema a resolver, y enumeración de las fuerzas a equilibrar (requisitos no funcionales como eficiencia, portabilidad, cambiabilidad).
3. **Solución:** conjunto de medidas que se han de tomar, como crear alguna clase, atributo o método, nuevos comportamientos entre clases.

Comprender qué son el contexto, el problema y la solución dentro de un patrón de diseño de software es clave para entender cómo y cuándo aplicarlo correctamente.

PATRONES ORIENTADOS A OBJETOS

Elementos básicos de un patrón

1. **Contexto:** situación en la que se presenta el problema de diseño.
2. **Problema:** descripción del problema a resolver, y enumeración de las fuerzas a equilibrar (requisitos no funcionales como eficiencia, portabilidad, cambiabilidad).
3. **Solución:** conjunto de medidas que se han de tomar, como crear alguna clase, atributo o método, nuevos comportamientos entre clases.

Comprender qué son el contexto, el problema y la solución dentro de un patrón de diseño de software es clave para entender cómo y cuándo aplicarlo correctamente.

PATRONES ORIENTADOS A OBJETOS

Elementos básicos de un patrón

1. **Contexto:** situación en la que se presenta el problema de diseño.
2. **Problema:** descripción del problema a resolver, y enumeración de las fuerzas a equilibrar (requisitos no funcionales como eficiencia, portabilidad, cambiabilidad).
3. **Solución:** conjunto de medidas que se han de tomar, como crear alguna clase, atributo o método, nuevos comportamientos entre clases.

Comprender qué son el contexto, el problema y la solución dentro de un patrón de diseño de software es clave para entender cómo y cuándo aplicarlo correctamente.

PATRONES ORIENTADOS A OBJETOS

Elemento	Pregunta que responde	Rol
Contexto	¿En qué situación ocurre esto?	Define el escenario
Problema	¿Qué desafío de diseño hay?	Identifica la necesidad
Solución	¿Cómo resolver de forma reutilizable y eficiente?	Propone una arquitectura de clases / objetos

Un patrón de diseño se puede considerar como un documento que define una estructura de clases que aborda una situación particular.

También debemos tener en cuenta que existen consecuencias positivas y negativas al usar patrones.

Consecuencias : Un patrón de diseño tiene desventajas y consideraciones asociadas a su propia aplicación. Esta sección ayuda a los desarrolladores a comprender las implicaciones de usar el patrón en diferentes contextos, ya sea como consecuencia o como limitación.

POO: CLASIFICACIÓN

Los patrones de diseño orientados a objetos según GoF, se agrupan por su propósito en tres grandes categorías:

- ❖ **Creacionales**
- ❖ **Estructurales**
- ❖ **Comportamiento**



Cada una con un enfoque distinto respecto al diseño del software. Varían en complejidad, nivel de detalle y escala de aplicabilidad.

POO: CLASIFICACIÓN- CREACIONALES

Se centran en cómo crear objetos de manera flexible, controlada y desacoplada. Abordan problemas relacionados con la instanciación de objetos, especialmente en contextos donde la creación directa (con new) no es conveniente o adecuada.

Problemas que resuelven:

- ❖ Complejidad en la creación de objetos complejos
- ❖ Necesidad de ocultar detalles de implementación
- ❖ Control sobre el número de instancias creadas

POO: CLASIFICACIÓN- CREACIONALES

Abstraen la forma en la que se crean los objetos, permitiendo la creación de las clases de forma genérica dejando para más tarde la decisión de qué clase crear/instanciar o cómo crearlas/instanciarlas.

CLASIFICACIÓN:

Patrones de creación de clase: la decisión se toma en los constructores de las clases y usan la herencia para determinar la creación de las instancias.

Patrones de creación de objetos: se modifica la clase desde el objeto.

POO: CLASIFICACIÓN- ESTRUCTURALES

Se enfocan en cómo componer objetos y clases para formar estructuras flexibles y reutilizables, respetando principios como el bajo acoplamiento y alta cohesión. Ayudan a definir relaciones claras entre entidades del sistema.

Problemas que resuelven:

- ❖ Composición y organización eficiente de clases/objetos
- ❖ Adaptación de interfaces incompatibles
- ❖ Añadir comportamiento o estructura sin modificar código existente

POO: CLASIFICACIÓN- COMPORTAMENTALES

Se centran en cómo los objetos interactúan entre sí, cómo se reparten las responsabilidades y cómo se comunican de manera efectiva. Son clave para organizar la lógica del sistema y mejorar su extensibilidad.

Problemas que resuelven:

- ❖ Distribución de responsabilidades entre objetos
- ❖ Comunicación desacoplada y flexible
- ❖ Implementación de flujos de control complejos

POO: CLASIFICACIÓN



POO: CLASIFICACIÓN



Proposito	Ámbito	
	Clases	Objetos
Creación	Factory Method	Abstrac Factory
		Builder
		Prototype
		Singleton
Estructurales	Adapter (de clase)	Adapter (de objetos)
		Bridge
		Composite
		Decorator
		Facade
		Flyweight
Comportamiento		Proxy
	Interpreter	Chain of Responsibility
	Template Method	Command
		Iterator
		Mediator
		Memento
		Observer
		State
		Strategy
		Visitor

POO: CLASIFICACIÓN

Característica	Creacionales	Estructurales	Comportamentales
Enfoque	Cómo se crean los objetos	Cómo se organizan / clasifican	Cómo interactúan entre sí
Nivel	Inicial (instanciación)	Medio (composición)	Alto (flujo de ejecución, lógica)
Beneficio clave	Control de instancias, flexibilidad	Bajo acoplamiento, reutilización	Flexibilidad, mantenimiento, extensión
Impacto	Instanciación del sistema	Estructura interna	Lógica y dinámica del sistema

PDS RESUMEN

- ❖ Los patrones son soluciones probadas y comprobadas a problemas comunes que se presentan una y otra vez
- ❖ Ahorra tiempo en el diseño y evita errores comunes, ya que se basa en experiencias previas exitosas.
- ❖ No indican cómo programar algo, sino cómo estructurar el diseño para resolver un problema de manera eficaz y mantenible.
- ❖ Cada patrón tiene una estructura estándar de documentación. Facilita su estudio, comprensión y comunicación entre desarrolladores.
- ❖ Representan soluciones a un conjunto de problemas similares, no a un problema único y específico.
- ❖ Los componentes están poco interrelacionados. Cada componente tiene una única responsabilidad bien definida.
- ❖ Varios patrones pueden usarse en conjunto para resolver problemas complejos.